

稼働カレンダー — 機能設計

006_DD-FN バージョン 1。SCR-006、FN-037-040、REQ-070-075。WI-010 承認済みリビジョン 1、手順 8。英語版の日本語版。Backend 設計のみ、未実装。

1. 文書管理と参照

項目	内容
文書 ID / 版	006_DD-FN / 1
システム / サブシステム	ProductionManagementAI / マスタデータ
作業項目 / 作成者	WI-010 / Agent
作成 / 更新日	2026-10-02 / 2026-10-02
状態	レビュー提出、未実装

版	日付	作成者	変更
1	2026-10-02	Agent	Backend method、正確な解決・能力、transaction 結果設計初版

入力：[主 DD](#)、[API](#)、[DB](#)、[BD](#)、[要件](#)、[計画](#)。先行設計は承認済み。API が HTTP 項目・error、主 DD が UI 状態、本書が backend method を所有。DD-SPD は予約のみ、未作成。既存指示・マスタ algorithm は変更しない。

2. モジュールと method 一覧

計画 PlantCalendarService（Application）が型付 repository port を使用する。Infrastructure は EF/Npgsql 投影・transaction を実装。解決・能力は純粋な検証可能規則。Controller に SQL・scheduling logic を持たせない。名前は提案で、compile 済みファイルではない。全 method は Agent 作成・更新、2026-10-02。

No.	Method	呼出し元 / 機能	戻り値
M-01	MonthAsync	API-PC-01 / FN-037	CalendarResult<MonthResult>
M-02	DayAsync	API-PC-02 / FN-037/039	CalendarResult<DayResult>
M-03	WeeklyAsync	API-PC-03 / FN-038	CalendarResult<WeeklyPage>

No.	Method	呼出し元 / 機能	戻り値
M-04	SaveWeeklyAsync	API-PC-04 / FN-038	CalendarResult<MutationResult>
M-05	WithdrawWeeklyAsync	API-PC-05 / FN-038	CalendarResult<MutationResult>
M-06	SaveExceptionAsync	API-PC-06 / FN-039	CalendarResult<MutationResult>
M-07	RemoveExceptionAsync	API-PC-07 / FN-039	CalendarResult<MutationResult>
M-08	LineChoicesAsync	API-PC-08 / FN-037/040	CalendarResult<LineChoicePage>
M-09	ProductChoicesAsync	API-PC-09 / FN-040	CalendarResult<ProductChoicePage>
M-10	CapacityAsync	API-PC-10 / FN-040	CalendarResult<CapacityResult>
M-11	ExceptionHistoryAsync	API-PC-11 / FN-039	CalendarResult<ExceptionPage>
M-12	ResolveDay	M-01/02/10	DaySummary
M-13	FloorCapacity	M-10	正規 quantity string
M-14	ReadSnapshotAsync	全 read method	型付投影または失敗
M-15	WriteTransitionAsync	M-04/05/06/07	MutationResult または結果確定付き失敗

共通 utility、引数、値対応

Utility	用途 / 境界
IPlantClock + TimeProvider	既存 DateOf(utc)/TimeZoneld、捕捉 UTC。browser/SQL CURRENT_DATE を基準にしない
AppDbContext / EF Core / Npgsql	既存 stack、新 ORM/provider なし
Calendar input validator	承認 API の型付不変条件・正規化を再検証。strict parser は業務検証の代替でない
既存 Identity policy	対象照会前 Admin/Operator。利用者別 DB role なし
Invariant formatter	UUID/version/decimal/date の正規 string。float・locale 依存なし

公開 method は API 対応の型付 query/command と CancellationToken を受ける。Query は許可された解析済み項目のみ。Command は観測 long revision と任意 target UUID。Client SQL、is_current、commit_revision、置換版は受け付けない。純粋 helper の引数は節 4/5。I/O・取消副作用なし。外部 request data なし、祝日 provider・外部 HTTP なし。応答項目一覧は API 所有。ここでの対応は mask→曜日順 string、snapshot→API、null marker→下位規則、整数→正規小数 string、型付失敗→既存 code/status/writeOutcome。新規応答 property なし。

3. ReadSnapshotAsync ― 整合読取境界（M-14）

引数：型付 query、投影 delegate、CancellationToken。戻り値：型付 API 投影または CalendarFailure。単一読取 transaction を所有し、長期 cache は持たない。

手順	処理	呼出し
1	Request 取消と 20s deadline を結合、command 15s	Scoped repository 設定
2	Read-only REPEATABLE READ 開始、SET LOCAL statement_timeout=15000ms、lock_timeout=5000ms	EF transaction、parameter 設定
3	state id=1 を読んだ後 utc=TimeProvider.GetUtcNow()、plantToday=clock.DateOf(utc) を 1 回取得。保存 timezone と設定一致検証	State、clock
4	snapshotVersion があれば state revision と比較。page 2+ 必須は事前検証。不一致 CALENDAR_STALE	Validator
5	参照・規則・count/page を同一 snapshot の必要列で取得	各 read method
6	型付結果返却前に transaction 終了。全経路で dispose、command 設定復帰	Wrapper

未有効化 read は context activation/version null、暗黙初期曜日なし。Timezone 不一致 503、日付再解釈なし。各 read で日付捕捉は 1 回、後の midnight でも混在しない。Snapshot は最初の SQL から共有する。[PostgreSQL 分離](#) を参照。Write は別 READ COMMITTED 境界。要求間 tracking cache、lazy loading/N+1 なし。Count と items は transaction 終了前に取得。

4. ResolveDay ― 優先順位と時間（M-12）

引数：DateOnly、CalendarContext、任意ライン、同日の最新工場・ライン例外、適用週間 snapshot。戻り値 DaySummary。能力保存・過去マスタ時間の保証なし。

手順	処理	呼出し
1	有効化前・状態なしは Unavailable、hours null、source None、fallback []、editable=false	Context guard
2	最新未削除ライン例外、最新未削除工場例外、最新未取消適用週間の順に候補	投影
3	先頭が適用規則。候補なしは Unavailable。残りは優先順 fallback、最大 2	Source mapper
4	例外状態または月曜起点 mask を確認。休業は "0" / Closed	曜日対応
5	明示稼働は hours/Explicit。それ以外は現ライン時間/LineCurrent、ラインなし null/LineDependent	ライン投影
6	保存・作成 editable は範囲内、本日以降、工場または稼働ライン。適用元 ID・理由保持	API mapper

同対象の current が削除・取消でも旧版を復活させない。週間は is_current AND NOT is_withdrawn、effective_from<=date の最大開始日。後の開始日取消は前の開始日を復帰し、取消した同日付の旧版を使わない。曜日 index=((int)date.DayOfWeek+6)%7、月 bit 0、日 bit 6、(mask & (1<<index)) !=0。Mask 0 は全休業で有効。ライン例外は

全体適用し、継承時間はライン基準。下位工場が明示時間でも混ぜない。過去継承時間は現設定と表示し、過去能力としない。停止ライン `editable=false` は保存・作成制限であり、本日・未来の既存削除は M-07 で独立判定する。

5. FloorCapacity — 正確な保守的算術（M-13）

引数：有効な decimal H 時間（0..24、小数 3 桁以下）、T 分/単位（0 超、999999999.999 以下、小数 3 桁以下）、承認単位。戻り値 invariant quantity string。保存不正値は不変条件失敗で、clamp しない。呼出し側が先に適格性判定。 $h=H\ 1000$ 、 $t=T\ 1000$ を丸めなしの整数へ変換。検証済み scale により整数。 $N=\text{checked}(h*60)$ 。

単位	計算	戻り値
個 / 本 / 枚 / 台 / セット	$q=N \text{ div } t$	非負整数 string
kg / m	$q3=\text{checked}(N*1000) \text{ div } t$	$q3/1000$ の正確な小数、不要末尾 0 除去

非負整数 div は floor を正確に実現する。.NET decimal 除算後 floor は中間丸めで境界を越え得るため禁止。 $h\leq 24000$ 、 $N\leq 1440000$ 、 $N*1000\leq 1440000000$ 、 $t\leq 999999999999$ 。すべて Int64。Checked 変換・乗算を使い double/float 禁止。最大能力は 24 時間・0.001 分で 1440000。有効な休業組合せは 0。

時間	分/単位	単位	期待数量 / 意図
4	0.125	個	"1920"、再稼働例
0.001	0.007	個	"8"、非終端比の floor
0.001	0.007	kg	"8.571"、正確な 3 桁 floor
1	60.001	個	"0"、整数境界直下
0.001	60	m	"0.001"、最小 quantum
0.001	60.001	m	"0"、quantum 直下、切上げ禁止
24	0.001	kg	"1440000"、最大

設計 vector の独立算術確認は文書証拠で、実装 C# helper の test ではない。Test ID は後で割当。

6. WriteTransitionAsync — 原子的変更境界（M-15）

引数：正規化 command、操作種別（週間保存・取消、例外保存・削除）、任意 line UUID、CancellationToken。MutationResult または API NotApplied/Unknown 付き失敗を返す。Retry loop なし。

手順	処理	呼出し
1	API deadline/timeout、READ COMMITTED、lock 前静的検証	Transaction
2	ライン例外は先にライン SELECT FOR SHARE、未知 NOT_FOUND。状態→ライン禁止	Parameterized query
3	State id=1 FOR UPDATE、未有効化 CALENDAR_NOT_ACTIVATED、timezone と観測版比較	State query
4	Marker 含む完全対象・日付の current を読み、submitted 対象識別・最新版 ID 検証	操作述語
5	全 lock 待機後に UTC と DateOf を再捕捉。有効化・本日未来・初回保護・稼働ラインを再確認。許可削除は停止可	Clock、規則
6	版・対象・認可検証後に正規化保存内容比較。同じ通常内容は changed=false、版加算なし	比較
7	Checked next=revision+1。既存 current を ID/is_current=true で旧版化（必ず 1 行）、next の不変 snapshot/marker INSERT	列更新・insert
8	State revision=next、updated_at_utc=capturedUtc、id=1 AND revision=observed（必ず 1 行）	State writer
9	COMMIT 送信直前 CommitIssued、応答確認 CommitConfirmed 後だけ changed=true	Commit

No-op も transaction を完了し、結果確定規則を守る。Snapshot・版・時刻は追加しない。新規対象は旧版化 update 不要。Created/update UTC と日付基準は同じ捕捉時刻から生成。検証後の時計進行で受入れた操作基準を変更しない。Statement ごとに today を読み直さない。

ロック SQL 意図（設計のみ）：

```
SELECT id, is_active FROM public.production_lines WHERE id = @lineId FOR SHARE;
SELECT id, activated_on, time_zone_id, revision
FROM public.plant_calendar_state WHERE id = 1 FOR UPDATE;
```

停止は非 key 更新なので FOR KEY SHARE へ置換せず FOR SHARE で競合させる。既存指示・マスタ path は calendar を取得せず、calendar も製品・対応製品・指示を lock しない。ライン→状態は既存製品→ライン→対応製品の逆 edge を導入しない。[PostgreSQL row lock](#) を参照。Transaction 終了まで保持し、中間 SaveChanges transaction/commit なし。Advisory lock、外部呼出し、数量保存、calendar delete trigger なし。

7. Endpoint 対応 method

引数は承認 API 項目だけ。全 method は CalendarResult<T>、value/failure の片方だけ返す。Parser/auth error は API。M-14/M-15 共通境界を各 method で重複しない。

M-01 MonthAsync

引数 MonthQuery(month,lineId?), ct。MonthResult。

手順	処理	呼出し
1	実月検証、DaysInMonthと日番号を使用。最終日 AddDays overflow 回避	Input rules
2	State・任意保持ライン、月初より前の最大 live 週間開始、月内 current live 開始取得	ReadSnapshotAsync
3	工場・選択ラインの current 例外を marker 含め一括取得、対象/日付辞書	Repository
4	28..31 日を memory predecessor cursor で昇順解決	ResolveDay

ライン月あたり最大 32 live 週間 head、62 例外 head。履歴全走査・日別 query なし。Data read は state・任意 line・前週間・月内週間・例外、最大 5。Runtime 性能保証ではない。

M-02 DayAsync

引数 DayQuery(date,lineId?), ct。DayResult。

手順	処理	呼出し
1	State・保持ライン・有効週間・最新工場/ライン取得	ReadSnapshotAsync
2	日を解決し、完全対象の latest exception を削除 marker 含め公開。未設定 null	ResolveDay、mapper
3	捕捉 context、過去制限。能力は計算しない	API mapper

M-03 WeeklyAsync

引数 WeeklyQuery(from,to,view,page,snapshotVersion?), ct。WeeklyPage。

手順	処理	呼出し
1	366 日以内、page 10000 以下、snapshot token	Input rules、ReadSnapshotAsync
2	範囲内開始日。Current は取消含め is_current、History は保持版	Repository
3	Count/page 20、effective_from DESC/commit_revision DESC/id ASC。from より前の最新 live を別取得	Repository
4	applicableBeforeFrom と context/version。空・超過 page は実総数	Mapper

M-04 SaveWeeklyAsync

WeeklySave(effectiveFrom,version,targetRevisionId?,workingDays), ct。MutationResult。重複なし曜日を mask に対応、[] は 0。

手順	処理	呼出し
1	曜日・対象日正規化検証	Input rules
2	Global lock 内、null は head なし、nonnull は同日 current（取消 marker 含む）	WriteTransitionAsync
3	過去開始拒否、本日置換で古い開始維持。再追加・修正は未取消 snapshot 追加	Weekly rules

M-05 WithdrawWeeklyAsync

WeeklyWithdraw(effectiveFrom,version,targetRevisionId), ct。MutationResult。

手順	処理	呼出し
1	Lock 内で同日 current 未取消対象	WriteTransitionAsync
2	初回有効化と本日以前を拒否。取消 no-op なし	Weekly rules
3	mask null の取消版追加。次読取で前開始日に復帰	Writer

M-06 SaveExceptionAsync

ExceptionSave(version,targetRevisionId?,lineId?,date,isWorking,hours?,reason?), ct。MutationResult。

手順	処理	呼出し
1	正確な時間・理由、休業・継承整合性、暗黙 0・丸めなし	Input rules
2	Line→state。null 対象は head なし、marker ID は再追加、normal latest ID は同対象日編集	WriteTransitionAsync
3	稼働ライン、有効化以降の本日・未来。is_removed=false の通常版追加	Exception rules

M-07 RemoveExceptionAsync

ExceptionRemove(version,targetRevisionId,lineId?,date), ct。MutationResult。

手順	処理	呼出し
1	停止可の参照 line→state。Current 未削除完全対象必須	WriteTransitionAsync
2	範囲内本日・未来。未存在・削除済み・不一致を成功としない	Exception rules
3	is_working/hours/reason null の削除版。元理由・全記録保持	Writer

M-08 LineChoicesAsync

ChoiceQuery(q?,page), ct。LineChoicePage。

手順	処理	呼出し
1	Context と active line、q は code/name literal contains	ReadSnapshotAsync
2	Backslash/percent/underscore escape、明示 escape の parameterized ILIKE。Count/page 50 code/id	既存 query pattern
3	現名称・時間・状態のみ、編集版なし。未有効化でも選択可	Mapper

M-09 ProductChoicesAsync

ProductChoiceQuery(lineld,q?,page), ct。ProductChoicePage。

手順	処理	呼出し
1	必須 line、未知 404、停止は real context と空	ReadSnapshotAsync
2	Active pair/product join、confirmed_unit と confirmed_unit_revision が現製品一致。SKU/name literal	Repository
3	Count/page 50 SKU/id、現 unit generation。暗黙再確認なし	Mapper

M-10 CapacityAsync

CapacityQuery(lineld,productId,date), ct。CapacityResult。

手順	処理	呼出し
1	Context と必須 line/product、未知 404、捕捉本日より過去 400	ReadSnapshotAsync
2	任意 pair・有効週間・工場/ライン例外。Pair 不在と必須 ID 不在を区別	Repository
3	日解決、API と同一 unavailable 優先順位	ResolveDay、eligibility
4	有効なら現係数と正確数量。無効は quantity/minutes null、架空 0 なし	FloorCapacity、mapper

優先順位：NotActivated→LineRetired→ProductRetired→PairMissing→PairRetired→UnitStale→CalendarUnavailable。確認単位または revision 不一致で UnitStale。ABA も含む。停止・stale pair は休業でも不可。過去能力は計算しない。有効休業は Available と "0"。混在単位、過去再計算、割当、開始制限なし。Pair 不在を inner join で消して誤った NOT_FOUND にしない。個別投影または安全な left join で必須 ID を明示検証。

M-11 ExceptionHistoryAsync

ExceptionHistoryQuery(date,lineId?,page,snapshotVersion?), ct。ExceptionPage。

手順	処理	呼出し
1	Context・任意 line・token。過去・停止参照可	ReadSnapshotAsync
2	同対象日の全版、marker 含め count/page。commit_revision DESC/id ASC	Repository
3	Page 非依存 current head。観測であり当該 client commit の証明ではない	Mapper

Nullable line は plant-null 分岐または null-safe equality、line_id = NULL を使わない。Page offset は API 上限と checked 算術。長期履歴は paging、暗黙切捨てなし。

8. 結果確定、予算、telemetry

内部 phase は BeforeCommit、CommitIssued、CommitConfirmed、RollbackConfirmed。新 API 項目ではない。COMMIT 送信前に phase 設定。既知検証・対象失敗は rollback 後 NotApplied。Deadline/deadlock/lock の rollback 応答確認で CALENDAR_BUSY/NotApplied。Commit 中に結果確定不可なら CALENDAR_WRITE_UNKNOWN/Unknown、Retry-After なし。一般 write 500 は API 通り Unknown。Commit acknowledgement まで成功にしない。応答配送喪失では client は Unknown になり得る。

Write retry strategy、savepoint replay、冪等台帳なし。取消だけで rollback 証明にしない。既に取消済み request token を cleanup に再使用せず、最大 5s の cleanup で rollback/dispose を試す。Cleanup は commit deadline 延長ではない。結果不明なら Unknown 維持、connection dispose、要求再開なし。中断 read は部分成功にしない。

永続化失敗	対応 / 境界
Named current/commit unique index 23505	Rollback 確認、409 CALENDAR_TARGET_CHANGED。他の予想外一意違反は UNEXPECTED
対象不在・旧版化/state update 0 行	Rollback、確認条件に対応 NOT_FOUND または CALENDAR_STALE/TARGET_CHANGED
Commit 前 lock/deadlock/cancel	既知 rollback だけ BUSY/NotApplied、他は安全な不確実性
保存 range/mask/marker 不正	UNEXPECTED、修復・clamp・架空能力なし
Checked revision overflow	Rollback、UNEXPECTED。循環・reset なし、owner 前進修正

承認 API の operation 20s、command/statement 15s、lock 5s、client 25s。SET LOCAL は機能 transaction 内だけ、EF per-context timeout 復帰と dispose。Pooled connection に設定を残して旧機能へ影響させない。Deadline は projection・算術・commit を含む。全操作で request 取消を伝播。

ActivitySource/Meter ProductionManagementAI.PlantCalendar を API 起動時に登録。上位 span plant-calendar.<handler>、handler は API の 11 名。子は plant-calendar.read、plant-calendar.resolve、plant-calendar.write。Tag は有限 stage/outcome、生値なし。純粋能力は resolve 内で計測、数量・ID・版の多次元 label なし。

Feature-aware middleware/adapter が要求最終 metric を 1 回だけ所有し、plant_calendar_operations_total、plant_calendar_duration_ms を operation/承認 outcome で記録。Service は結果提供のみで二重計数しない。401/403/parser も 1 回。CommitIssued 後の結果不明は cancelled より unknown 優先。能力 Unavailable は 200 でも unavailable。Log は operation/outcome/traceld のみ。SQL・日付・search・理由・係数・token・認証情報なし。Exporter 主張なし。

9. 検証観点と次レビュー

要件	後の実装検証
REQ-070	実月・閏日・9999 年、snapshot、未有効化、N+1 なし
REQ-071	全休業 mask、最新取消と前開始日、本日追加、保持修正
REQ-072	Null 工場重複、再追加、休業・継承、Unicode 理由、Cancel 無変更
REQ-073	全体優先・再稼働、整数 floor 境界、ABA・停止、無効と休業 0
REQ-074	ロール、別対象全体競合、lock/midnight、0 行 rollback、COMMIT 不明、cleanup、単一 metric
REQ-075	指示・マスタ書込、納期・期間・開始 algorithm 変更なし

未決業務事項・API/DB 拡張なし。整数算術、投影 query、原子的述語、commit phase、cleanup を FN レビューで判断する。C#・SQL lock・race・telemetry integration は未実装・未検証。全 gate は SPD と手順 11 後。本書と EN/JA PDF を提出し、承認後に DD-SPD を作成する。承認済み入力不変、本フェーズの migration/code/外部操作なし。